Ending ? "What has beginnings has endings, and what has code, has flags"

Flag: TVX{S0me_1nt2r2st1ng_t2xt_h2r2}

Challenge (as given): a single Windows executable, Ending.exe
(10,752 bytes, SHA-256
16686a047bb948a327139cb1d80fc2a62b8e1f85b551645a4b34073d4fa7f3e2),
with the tagline above and "find the flag". No flag format was stated;
the binary itself defines the expected string.

Intuition

The tagline is the map: the program has a *beginning* layer you can see
(a Base64 string), an *ending* you have to compute (a virtual machine's
output), and the flag exists only as the VM's behaviour ? it is never
stored anywhere in the file.

Step-by-step analysis

1. File identification

file reports a PE32 console executable, and specifically a Mono/.NET
assembly: 3 sections (.text, .rsrc, .reloc), entry point is the
_CorExeMain stub. Disassembly with monodis shows one class, Program,
with four methods ? Ror, Unpack, ReadWord, Run, Main ? and one
literal field, package: a 1,308-character Base64 string.

2. Layer 1 ? the package

Per the IL, Unpack() does:

1. Convert.FromBase64String(package) ? bytes starting 1F 8B (gzip).
2. GZipStream decompress ? 958 bytes of high-entropy data.
3. A second, hand-rolled transform on every byte i:

```
b[i] = ROR8(b[i], (i % 5) + 1) XOR ((199 + 91*i) & 0xFF)
```

That output is not text ? it is bytecode.

3. Layer 2 ? the virtual machine

Run() interprets the bytecode with:

- 4 registers, UInt16 each; 128 memory cells, UInt16 each
(ReadWord fetches little-endian 16-bit immediates).
- A 31-byte output buffer ? the flag is 31 characters.
- Opcode 0xFF halts (the "ending").

Opcode	Meaning

```
| 0x10 / 0x11 / 0x12 | R[a] = imm16 / R[a] ^= imm16 / R[a] += imm16 |
| 0x13 | R[a] = ROL16(R[a], b) |
| 0x14 / 0x15 | R[a] = M[b] / M[a] = R[b] |
| 0x16 / 0x17 / 0x18 | R[a] ^= R[b] / R[a] += R[b] / R[a] -= R[b] |
| 0x19 | R[a] = ~R[a] (single operand, re-read) |
| 0x1A | R[a] = M[R[b]] |
| 0x30 | jump if R[a] != 0, signed-byte offset |
| 0x31 | unconditional jump, signed-byte offset |
| 0x40 | output low byte of R[a] |
| 0xFF | halt |
```

The one trap in the VM semantics is the jump base. For 0x30 the IL adds the offset to ip *after* the offset byte has been fetched; for 0x31 it adds it to the offset byte's own position. Getting that single byte wrong derails execution into a bad opcode (0xBE at 0x379) within a few hundred steps ? a useful smoke signal that the semantics, not the data, are off.

4. Running it

Re-implementing Unpack() plus the VM in Python (stdlib only) and executing the 958-byte program produces 31 output bytes:

TVX{S0me_1nt2r2st1ng_t2xt_h2r2}

Leet-decoded it reads "some interesting text here" (0=o, 1=i, 2=e) ? but the digits are load-bearing, not a typo in the solver.

5. Verification against the real binary

Main() prompts flag> and string-compares the input with the VM output. Running the genuine binary under Mono:

- TVX{S0me_1nt2r2st1ng_t2xt_h2r2} ? accepted
- the "corrected English" variant TVX{S0me_1nterest1ng_text_here} ? rejected
- a random string ? rejected

Solver

solvers/solve_01_ending.py parses the PE's #US heap to extract the package literal (no .NET tooling needed), applies the Base64 ? GZip ? ROR/XOR unpacking, emulates the VM exactly, and prints the flag. solvers/solve_all.py runs it. Fresh-unzip test: 1/1.

```
unzip ending_writeup.zip && cd ending_writeup && python3 solvers/solve_all.py
```

Lesson

For "the flag is computed, not stored" binaries, disassemble first and let the checker's own semantics be the spec ? then verify the recovered string against the real binary, including a near-miss variant, so a plausible-looking leet reading can't silently replace the true flag.